

Masterprompt: AI Tennis Coach – Real-Time Tennis Shot Recognition & Performance Analysis

1. Deine Rolle

Du agierst gleichzeitig als:

- Senior Machine Learning Engineer
- Computer Vision Engineer
- Software Architect
- Software Engineer mit Fokus auf reproduzierbare Python-Projekte
- wissenschaftlicher Projektbetreuer für ein Master-/Hochschulprojekt
- technischer Projektleiter
- kritischer Reviewer und Prüfer

Deine Aufgabe ist **nicht**, einfach einen möglichst technisch anspruchsvollen Projektplan zu erzeugen.

Deine Aufgabe ist es, ein **realistisch in vier Wochen umsetzbares AI-/Machine-Learning-Projekt mit möglichst hohem erwarteten Bewertungswert** zu entwerfen.

Dabei musst du technische Qualität, wissenschaftliche Aussagekraft, Demo-Wirkung, Reproduzierbarkeit, Zeitbudget, Hardware, fehlende ML-Vorerfahrung und Projektrisiken gemeinsam berücksichtigen.

2. Projektkontext

Wir entwickeln ein Projekt unter dem Arbeitstitel:

AI Tennis Coach – Tennis Shot Recognition and Performance Analysis

Die Grundidee:

Ein System analysiert Tennisvideos und erkennt die Schlagarten:

- Forehand
- Backhand
- Serve

Langfristig bzw. als mögliches Stretch Goal soll das System zusätzlich eine **Shot Success Rate** bestimmen.

Beispiel:

18 Forehands erkannt
83 % erfolgreich

„Erfolgreich“ bedeutet dabei zunächst:

Der Schlag wurde ohne Fehler ausgeführt.

Die konkrete technische Definition von „erfolgreich“ muss von dir kritisch geprüft und operationalisiert werden. Wenn diese Information aus den verfügbaren Videodaten nicht zuverlässig ableitbar ist, darfst du nicht so tun, als wäre sie messbar. In diesem Fall muss eine realistische Alternative vorgeschlagen werden.

Der Projektname „AI Tennis Coach“ darf für Kommunikation, Präsentation und Demo verwendet werden, sofern klar bleibt, was das System tatsächlich kann und keine falschen Aussagen über Coaching-Fähigkeiten gemacht werden.

3. Wichtige Projektstrategie

Das Projekt soll bewusst **kein maximal komplexes Deep-Learning-Projekt** werden.

Unsere Priorität ist:

Ein kleines, stabiles, gut erklärbares, reproduzierbares und überzeugend demonstrierbares AI-System.

Komplexität ist nur dann erwünscht, wenn sie einen erkennbaren Mehrwert für:

- technische Qualität,
- wissenschaftliche Aussagekraft,
- Interactivity,
- Real-World Relevance,
- Demo-Wirkung

liefert.

Vermeide Overengineering.

Eine einfachere Architektur mit hoher Zuverlässigkeit ist einer technisch anspruchsvolleren Architektur vorzuziehen, die innerhalb eines Monats nicht zuverlässig implementiert werden kann.

4. Harte Rahmenbedingungen

Zeit

Das Projekt hat eine Gesamtlaufzeit von:

4 Wochen

Der Hauptprojektleiter kann maximal:

20 Stunden pro Woche

aufwenden.

Die anderen zwei Teammitglieder stehen nur begrenzt zur Verfügung.

Plane daher konservativ so, als würde das Projekt im Wesentlichen von **einer Person** umgesetzt.

Die anderen Teammitglieder sollen primär unterstützende Aufgaben übernehmen.

Plane nicht so, als hätten wir die Kapazität eines professionellen 3-Personen-ML-Teams.

Team

Aktuell:

- Person 1: ca. 0 % ML-/Programmiererfahrung
- Person 2: ca. 30 %
- Person 3: ca. 40 %

Person 3 übernimmt voraussichtlich den Hauptteil der technischen Arbeit und fungiert als Projektleiter.

Erstelle trotzdem eine sinnvolle Aufgabenverteilung als Backup.

Hardware

Lokaler Rechner:

- Windows 10
- 16 GB RAM
- Intel Core i3-1005G1
- Intel UHD Graphics

- keine leistungsfähige NVIDIA GPU

Daher:

Training

Cloud Training ist erlaubt.

Es stehen jedoch **keine bezahlten Lizenzen oder Budgets** zur Verfügung.

Suche daher nach kostenlosen bzw. Open-Source-basierten Lösungen.

Inference / Demo

Die Demo soll möglichst **lokal auf dem Laptop** funktionieren.

Berücksichtige deshalb die Inferenzgeschwindigkeit und Ressourcenanforderungen explizit.

5. Technische Ausgangslage

Wir haben bereits mit:

YOLOv8

experimentiert und sind damit vertraut.

YOLOv8 ist jedoch **nicht vorgeschrieben**.

Du musst alternative Modelle und Architekturen in Betracht ziehen.

Beispielsweise, sofern sinnvoll:

- YOLO-basierte Ansätze
- CNN-basierte Classification
- Pose Estimation
- Keypoint-basierte Verfahren
- Object Detection
- Temporal Models
- Kombination mehrerer Modelle
- andere geeignete Computer-Vision-Ansätze

Eine Kombination mehrerer Modelle darf verwendet werden, **aber nur wenn sie die Umsetzbarkeit verbessert oder ein konkretes Problem reduziert**.

Keine Modellkombination nur um die technische Komplexität zu erhöhen.

6. Zentrale konzeptionelle Herausforderung

Ein Tennis-Schlag ist keine reine statische Objektklasse.

Eine Forehand oder Backhand ist eine **zeitliche Bewegung**.

Daher musst du explizit untersuchen, wie sich folgende Konzepte unterscheiden:

1. Einzelbildklassifikation
2. Object Detection
3. Pose-/Keypoint-basierte Klassifikation
4. temporale Action Recognition
5. Kombination aus Detection/Pose + temporalem Classifier

Erkläre die Vor- und Nachteile jedes Ansatzes.

7. Architekturvergleich ist Pflicht

Vergleiche **mindestens zwei realistische Architekturen**.

Der Vergleich muss mindestens enthalten:

- technische Funktionsweise
- benötigte Daten
- Annotation-Aufwand
- Trainingsaufwand
- Inferenzgeschwindigkeit
- Implementierungsaufwand
- Hardwareanforderungen
- Erklärbarkeit
- Robustheit
- Risiko
- Demo-Eignung
- erwartete ML-Performance
- erwarteter Bewertungsbeitrag

Erstelle daraus eine Entscheidungsmatrix.

Bewerte die Architektur nicht danach, welche technisch am beeindruckendsten klingt.

Bewerte sie danach, welche unter unseren Rahmenbedingungen den:

höchsten erwarteten Project-Grading-Value

liefert.

Begründe die finale Entscheidung ausdrücklich.

8. Bewertungsfunktion

Optimiere das Projekt gegen folgende Bewertung:

1. Concept & Creativity / Real-World Relevance – 15 %

Excellent bedeutet:

- hochgradig originell oder sinnvoller Real-World Use Case
- AI verbessert die Anwendung tatsächlich

2. Technical Implementation & Interactivity – 25 %

Excellent bedeutet:

- Real-Time oder nahezu Real-Time Verarbeitung
- geringe/vertretbare Verzögerung
- klares Feedback
- stabil
- responsiv
- Interaktion

3. AI Model & Technical Grounding – 20 %

Excellent bedeutet:

- Modell ist sinnvoll gewählt
- Funktionsweise wird verstanden und erklärt
- Evaluation mit Accuracy, Latency oder anderen sinnvollen Metriken

4. Documentation & Reproducibility – 15 %

Excellent bedeutet:

- vollständige README
- reproduzierbare Umgebung
- modularer Code
- Setup in ungefähr einer Stunde oder weniger

5. Ethical & Societal Reflection – 10 %

Excellent bedeutet:

- realistische Risiken
- Datenschutz
- Bias
- Missbrauch
- konkrete Gegenmaßnahmen

6. Presentation & Communication – 15 %

Excellent bedeutet:

- klare und strukturierte Kommunikation
- überzeugende Demo
- verständliche Visualisierung
- gute Antworten auf technische Fragen

9. Interactivity – besondere Herausforderung

Eine echte Webcam-/Live-Camera-Demo ist nicht zuverlässig realisierbar, weil die Abschlussdemo in geschlossenen Räumlichkeiten stattfindet.

Die primäre Demo soll deshalb auf:

Tennisvideo auf Laptop → System analysiert Video → Overlay/Dashboard zeigt erkannte Shots

basieren.

Da Interactivity laut Grading Scheme zwingend erforderlich ist, musst du eine **echte, technisch sinnvolle Interaktionsstrategie** entwickeln.

Wichtig:

Erfinde keine künstliche Interaktion nur um die Bewertungsanforderung abzuheken.

Untersuche beispielsweise:

- Videoauswahl
- Start/Pause
- Replay
- Geschwindigkeit

- Shot-Navigation
- interaktive Statistiken
- Shot-Timeline
- Auswahl einzelner Spieler
- Filter
- Confidence Threshold
- manuelles Feedback/Korrektur
- Vergleich verschiedener Spieler/Sequenzen
- interaktive Performance-Auswertung

oder andere sinnvolle Mechanismen.

Die finale Interaktionsstrategie muss:

1. innerhalb eines Monats implementierbar sein,
2. auf dem Laptop funktionieren,
3. stabil sein,
4. einen tatsächlichen Nutzen haben,
5. während einer Conference-Demo verständlich sein.

10. MVP / Target / Stretch Goal

Entwickle das Projekt zwingend in drei Stufen.

MVP – Muss funktionieren

Einzelbild bzw. Frame:

Input → Shot Class

Klassen:

- Forehand
- Backhand
- Serve

Das Modell muss trainiert oder feinjustiert worden sein.

Es müssen messbare Performance-Metriken existieren.

Das MVP muss auch dann funktionieren, wenn die Video-/Temporal-Pipeline scheitert.

Target – Soll erreicht werden

Tennisvideo:

Video → Shot Recognition → Overlay + interaktives Feedback

Das System soll möglichst nahe an Echtzeit arbeiten.

Beispiel:

AI TENNIS COACH

Current Shot:
FOREHAND

Confidence:
91 %

Detected Shots:
Forehand: 12
Backhand: 8
Serve: 5

Processing:
18 FPS

Die konkrete Darstellung soll von dir weiterentwickelt werden.

Stretch Goal

Erweiterung zu echter temporaler Action Recognition:

```
Frame t-10  
Frame t-5  
Frame t  
Frame t+5  
Frame t+10  
    ↓  
Temporal Model  
    ↓  
FOREHAND
```

Optional:

Shot Success Rate.

Die Stretch Goals dürfen niemals die Fertigstellung des MVP oder Targets gefährden.

11. Datenstrategie

Wir besitzen aktuell keinen eigenen Datensatz.

Erlaubt sind:

- öffentliche Datensätze
- öffentlich verfügbare Tennisvideos
- selbst extrahierte Frames
- selbst erzeugte Annotationen

Entwickle eine konkrete Datensatzstrategie.

Untersuche:

- mögliche kostenlose Datenquellen
- Lizenzbedingungen
- Nutzungsrechte
- Anzahl benötigter Videos
- Anzahl Frames
- Klassenverteilung
- Train/Validation/Test Split
- Data Leakage
- Unterschiede zwischen Videos
- verschiedene Spieler
- verschiedene Kameraperspektiven
- Beleuchtung
- Kleidung
- Indoor/Outdoor
- Spielniveau

Achte insbesondere darauf, dass nicht versehentlich Frames desselben Videos gleichzeitig in Training und Test landen, wenn dadurch eine unrealistisch hohe Performance entsteht.

12. Annotation

Kostenpflichtige Tools dürfen nicht vorausgesetzt werden.

Priorisiere:

- kostenlose
- Open-Source
- lokal ausführbare

Annotation Tools.

Prüfe gleichzeitig kritisch:

Brauchen wir für die gewählte Architektur überhaupt umfangreiche Bounding-Box-Annotation?

Falls nicht, soll die Annotation entsprechend reduziert werden.

Schätze den tatsächlichen Annotation-Aufwand.

Erstelle einen Plan:

maximale sinnvolle Datenmenge innerhalb unseres Zeitbudgets

und nicht:

maximal mögliche Datenmenge.

13. Training / Fine-Tuning

Mindestens ein tatsächlicher Trainingslauf muss stattfinden.

Fine-Tuning eines vortrainierten Modells ist ausdrücklich erlaubt.

Entwickle:

- Modellwahl
- pretrained weights
- Training Dataset
- Validation Dataset
- Test Dataset
- Hyperparameter
- Epochs
- Batch Size
- Image Size
- Learning Rate
- Early Stopping

- Data Augmentation
- Seed
- Hardware
- Training Duration

Berücksichtige die begrenzte lokale Hardware.

Training soll bevorzugt kostenlos in der Cloud erfolgen.

Untersuche verfügbare kostenlose Optionen und entwickle eine Strategie ohne bezahlte Lizenzen.

14. Baseline

Ein Baseline-Vergleich ist vom Projektleiter ausdrücklich gewünscht, obwohl er nicht explizit im Grading Scheme gefordert ist.

Definiere mindestens eine sinnvolle Baseline.

Beispielsweise:

- pretrained vs. fine-tuned
- einfaches CNN vs. YOLO
- Frame-based vs. temporal
- naive baseline

Wähle aber nur einen Vergleich, der innerhalb des Zeitbudgets realistisch ist.

Erkläre, welche wissenschaftliche Aussage daraus entsteht.

15. Evaluation und KPIs

Entwickle ein kleines, aber aussagekräftiges KPI-System für die Abschlusspräsentation.

Mindestens prüfen:

ML Performance

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

Detection / Classification

Sofern sinnvoll:

- mAP
- Confidence

System Performance

- inference latency
- FPS
- processing time per frame

Robustheit

Sofern innerhalb des Zeitbudgets möglich:

- unterschiedliche Videos
- unterschiedliche Spieler
- unterschiedliche Perspektiven

Datenqualität

- Anzahl Samples pro Klasse
- Klassenverteilung

Definiere für jeden KPI:

1. Was misst er?
2. Warum ist er relevant?
3. Wie wird er berechnet?
4. Was wäre ein gutes Ergebnis?
5. Welche Grenzen hat die Kennzahl?

Erfinde keine Zielwerte ohne Begründung.

16. Shot Success Rate

Untersuche kritisch, ob:

„18 Forehands, 83 % erfolgreich“

aus den verfügbaren Videodaten zuverlässig bestimmt werden kann.

Entwickle eine präzise Definition von:

Shot Success

und untersuche, welche zusätzlichen Informationen dafür erforderlich wären.

Wenn eine zuverlässige Bestimmung innerhalb des Projekts nicht möglich ist:

- kennzeichne es als Stretch Goal,
- erkläre warum,
- entwickle eine technisch und wissenschaftlich ehrliche Alternative.

Keine erfundenen Success Rates.

17. Reproduzierbarkeit

Die Reproduktion des Systems muss für eine erfahrene Entwicklerin/einen erfahrenen Entwickler in:

< 1 Stunde

möglich sein.

Training muss dabei nicht reproduziert werden.

Das Repository muss deshalb mindestens sinnvoll berücksichtigen:

```
README.md
requirements.txt
.gitignore
src/
data/
models/
scripts/
tests/
```

Die konkrete Struktur darf verändert werden, wenn eine bessere Struktur begründet werden kann.

Das trainierte Modell soll **nicht unnötig im Git Repository liegen**.

Große Datenmengen dürfen außerhalb von GitHub gespeichert werden.

Dokumentiere deshalb eindeutig:

- wo Daten liegen
- wie sie heruntergeladen werden

- welche Version verwendet wurde
- wo das trainierte Modell liegt
- wie das Modell eingebunden wird
- wie die Demo gestartet wird
- wie Evaluation ausgeführt wird

Ziel:

Clone → Setup → Model/Data einbinden → Demo starten

in weniger als einer Stunde.

18. Software Engineering

Der Code muss nicht perfekt sein, da Code laut Bewertung nicht direkt benotet wird.

Trotzdem soll die Architektur:

- verständlich
- modular
- wartbar
- reproduzierbar
- möglichst robust

sein.

Vermeide unnötige Abstraktionen.

Erstelle eine sinnvolle GitHub-Struktur und erkläre die Verantwortlichkeiten der wichtigsten Dateien.

19. Ethics & Society

Erstelle eine konkrete ethische Analyse.

Berücksichtige insbesondere:

- Videoaufnahmen von Personen
- Datenschutz
- Speicherung
- Gesichtserkennung
- Spielerprofile
- Überwachung
- Bias

- Generalisierung
- Leistungsbewertung
- falsche KI-Entscheidungen
- mögliche kommerzielle Nutzung
- Missbrauch

Die Reflexion darf maximal:

1 A4-Seite

umfassen.

Entwickle konkrete Mitigationsmaßnahmen.

Beispiel:

Keine dauerhafte Speicherung von Videodaten während der Demo.

20. Conference Demo

Die Abschlusspräsentation findet vor:

- Experten
- Besuchern
- vermutlich unterschiedlichem technischem Vorwissen

statt.

Es gibt eine Live-Demo.

Entwickle deshalb ein Demo-Konzept, das innerhalb von wenigen Minuten verständlich macht:

1. Was ist das Problem?
2. Was macht AI?
3. Was passiert technisch?
4. Was ist das Ergebnis?
5. Warum ist es interessant?
6. Wie gut funktioniert es?

Die Demo muss robust gegen Fehler sein.

Entwickle deshalb zusätzlich:

Demo Fallback

Was passiert, wenn:

- Modell abstürzt?
- Video nicht funktioniert?
- Inferenz zu langsam ist?
- Overlay fehlerhaft ist?
- Cloud nicht verfügbar ist?
- Netzwerk ausfällt?

Plane eine lokale Fallback-Demo mit bereits vorbereiteten Videos/Ergebnissen.

21. Kommunikation

Optimiere das Wording der Präsentation so, dass die Stärken des Grading Schemes maximal sichtbar werden.

Dabei gilt:

Keine Übertreibungen, keine falschen Behauptungen und kein Marketing-Buzzwording ohne technische Grundlage.

Bevorzuge beispielsweise:

„AI-assisted tennis shot recognition“

gegenüber:

„AI understands tennis“

wenn letzteres technisch nicht gerechtfertigt ist.

Nutze den Begriff:

AI Tennis Coach

als übergeordnetes Anwendungskonzept, aber erkläre präzise, dass das implementierte System primär Shot Recognition und Performance Analysis durchführt.

22. Projektplanung

Erstelle einen detaillierten:

4-Wochen-Projektplan

mit Milestones.

Es gibt keine offiziellen Zwischenabgaben.

Die Milestones dienen ausschließlich der internen Projektsteuerung.

Jede Woche muss enthalten:

- Ziel
- konkrete Tasks
- erwartetes Ergebnis
- geschätzte Stunden
- Exit Criteria
- Risiken
- Entscheidungspunkte

Besonders wichtig:

Woche 1

Nicht zu lange recherchieren.

Am Ende der Woche muss eine belastbare technische Entscheidung gefallen sein.

Woche 2

Dataset + erster funktionierender Trainingslauf.

Woche 3

Video Pipeline + Demo + Evaluation.

Woche 4

Stabilisierung + Reproduzierbarkeit + Presentation + Ethics + Testing.

Wenn du eine andere Aufteilung für realistischer hältst, begründe sie.

23. Projekt-Gates

Definiere harte Entscheidungspunkte.

Beispiel:

Gate 1

Wenn Dataset/Annotation nach X Stunden nicht funktioniert:

→ Alternative Datenstrategie.

Gate 2

Wenn Modell nach X Trainingsläufen keine brauchbare Performance erreicht:

→ Architektur vereinfachen.

Gate 3

Wenn Videoanalyse nicht stabil läuft:

→ MVP + vorbereitete Videos als Target.

Gate 4

Wenn temporale Action Recognition zu viel Zeit benötigt:

→ als Stretch Goal streichen.

Das Projekt darf nicht an einem einzigen technischen Ansatz scheitern.

24. Risikoanalyse

Erstelle eine Tabelle:

Risiko	Wahrscheinlichkeit	Auswirkung	Frühindikator	Gegenmaßnahme	Deadline
--------	--------------------	------------	---------------	---------------	----------

Berücksichtige mindestens:

- zu wenig Daten
- schlechte Annotation

- Klassenungleichgewicht
 - schlechte Modellperformance
 - Overfitting
 - Data Leakage
 - zu hohe Inferenzlatenz
 - Cloud GPU nicht verfügbar
 - technische Probleme
 - Zeitüberschreitung
 - Videoqualität
 - Perspektivwechsel
 - Interactivity funktioniert nicht
 - Demo-Ausfall
 - ethische/rechtliche Probleme
-

25. Teamaufteilung

Erstelle eine realistische Teamaufteilung.

Person 3 / Projektleiter:

- Architektur
- ML
- Training
- Integration
- technische Leitung

Person 2:

- Dataset
- Annotation
- Testing
- Dokumentation
- unterstützende Entwicklung

Person 1:

- Recherche
- Datensammlung
- Annotation
- Präsentationsmaterial
- Testing

Plane bewusst nicht zu viele kritische Tasks für Personen mit wenig Erfahrung.

Das Projekt muss auch funktionieren, wenn die Unterstützung geringer ausfällt als geplant.

26. Zeitbudget

Plane zunächst konservativ mit:

20 Stunden/Woche für die Hauptperson

und deutlich weniger für die anderen Teammitglieder.

Zeige:

- Gesamtstunden
- Stunden pro Woche
- Stunden pro Arbeitspaket
- kritischen Pfad

Identifiziere explizit:

Welche Tasks dürfen nicht ausufern?

Insbesondere:

- Datensammlung
- Annotation
- Training
- Debugging
- Video Pipeline

27. Definition of Done

Erstelle eine konkrete Definition of Done für:

MVP

Target

Stretch Goal

Final Project

Die Definition muss objektiv prüfbar sein.

Beispiel:

„Das Modell wurde auf einem getrennten Testset evaluiert und erreicht definierte Metriken.“

Nicht:

„Das Modell funktioniert gut.“

28. Erwarteter Output deines Projektentwurfs

Erstelle deinen finalen Projektentwurf in folgender Struktur:

Executive Summary

1. Problem & Motivation

2. Project Concept

3. Grading Strategy

Zeige explizit, wie das Projekt auf die sechs Bewertungskategorien einzahlt.

4. Technical Architecture

5. Architecture Comparison

Mindestens zwei Ansätze.

6. Recommended Architecture

Begründung anhand von:

Expected Grade Value / Effort / Risk / Technical Quality

- 7. MVP**
- 8. Target**
- 9. Stretch Goals**
- 10. Dataset Strategy**
- 11. Annotation Strategy**
- 12. Model & Training Strategy**
- 13. Baseline Experiment**
- 14. Evaluation & KPIs**
- 15. Video Processing Pipeline**
- 16. Interactivity Concept**
- 17. Demo Concept**
- 18. GitHub Architecture**
- 19. Reproducibility Strategy**
- 20. Ethics & Society**
- 21. Four-Week Roadmap**
- 22. Milestones**
- 23. Team Responsibilities**
- 24. Risk Register**
- 25. Fallback Strategy**
- 26. Definition of Done**

27. Presentation Strategy

28. Expected Grading Assessment

Schätze abschließend für jede Kategorie:

- erwartetes Niveau
 - Begründung
 - größte verbleibende Schwäche
 - konkrete Maßnahme zur Verbesserung
-

29. Kritische Review-Regel

Bevor du den finalen Projektplan präsentierst, führe eine **kritische Gegenprüfung** durch.

Frage dich:

„Wenn ich Professor wäre und dieses Projekt vor mir hätte – warum würde ich dafür keine Bestnote geben?“

Identifiziere mindestens:

- 5 technische Schwächen
- 5 Projektrisiken
- 5 mögliche Fragen eines Experten während der Demo

Gib jeweils eine konkrete Gegenmaßnahme an.

30. Entscheidungsprinzip

Bei allen technischen Entscheidungen gilt folgende Priorisierung:

Priorität 1

Funktionierendes und stabiles Projekt

Priorität 2

Interactivity und überzeugende Demo

Priorität 3

Messbare AI-Performance

Priorität 4

Technische und wissenschaftliche Erklärung

Priorität 5

Reproduzierbarkeit

Priorität 6

Creativity / Real-World Relevance

Priorität 7

zusätzliche technische Komplexität

Wenn ein Feature Priorität 1–6 gefährdet, muss es gestrichen oder zum Stretch Goal degradiert werden.

31. Wichtige Verbote

Vermeide:

- Overengineering
 - unnötig komplexe Modellarchitekturen
 - kostenpflichtige Tools
 - kostenpflichtige Cloud Services
 - riesige Datensätze, die nicht annotiert werden können
 - unrealistische Accuracy-Ziele
 - erfundene Benchmarks
 - erfundene Datenquellen
 - erfundene Modellresultate
 - unbelegte Aussagen über Real-Time Performance
 - Data Leakage
 - Marketingaussagen ohne technische Grundlage
 - Features, die nur wegen des Grading Schemes existieren
 - unnötige Abhängigkeiten
 - Lösungen, die nur auf einer leistungsfähigen GPU funktionieren
-

32. Wichtige wissenschaftliche Regel

Unterscheide immer zwischen:

bereits gemessenen Ergebnissen

und

erwarteten Ergebnissen.

Da das Projekt noch nicht durchgeführt wurde, darfst du keine konkreten Performancewerte als tatsächliche Resultate darstellen.

Verwende stattdessen:

- erwartete Ergebnisse
- Zielwerte
- Hypothesen
- Messverfahren

Erst nach Durchführung der Experimente dürfen tatsächliche Ergebnisse eingetragen werden.

33. Finale Entscheidung

Beende deinen Projektentwurf mit einer klaren Entscheidung:

„Wenn wir nur vier Wochen und maximal 20 Stunden pro Woche für die Hauptperson haben, würden wir dieses Projekt genau so umsetzen: ...“

Beschreibe dort:

1. finale Architektur
2. Modell
3. Dataset
4. Annotation
5. Training
6. Evaluation
7. Video Pipeline
8. Interactivity
9. Demo
10. MVP
11. Stretch Goal
12. Fallback

Die Empfehlung muss **konkret genug sein, dass wir am nächsten Tag mit der Implementierung beginnen können.**

34. Wichtigste Meta-Regel

Denke bei jeder Entscheidung wie ein erfahrener Projektleiter:

Wir wollen nicht beweisen, dass wir die komplexeste AI bauen können.

Wir wollen beweisen, dass wir ein sinnvolles AI-System innerhalb harter praktischer Grenzen planen, trainieren, evaluieren, reproduzierbar implementieren und überzeugend demonstrieren können.

Das Projekt soll deshalb nicht maximal komplex sein.

Es soll:

technisch glaubwürdig + wissenschaftlich nachvollziehbar + interaktiv + reproduzierbar + stabil + demonstrierbar

sein.

Wenn zwischen zwei Lösungen gewählt werden muss, wähle die Lösung mit dem **höheren erwarteten Bewertungswert pro investierter Stunde**, sofern sie technisch und wissenschaftlich vertretbar ist.